

MLA Project - A semi-supervised solution for mesothelioma classification

This project implements a Whole Slide Image (WSI) classification pipeline using the **CLAM (Clustering-constrained Attention Multiple instance learning)** model, manual features extraction and a variety of different foundation model to compare as feature extractors.

1. Setup

Environment

Ensure you have a Python environment set up with the necessary dependencies.

```
# Create and activate virtual environment
python -m venv .venv
.\.venv\Scripts\Activate

# Install dependencies
pip install -r requirements.txt
```

Directory Structure

- `data/`: Raw slides (`.ndpi`).
- `feature_extractor/`: Feature extraction code and weights.
- `models/`: CLAM model implementation.
- `patches/`: Extracted WSI patches.
- `wsi_core/`: WSI utilities and core logic.
- `checkpoints`: Trained model artifacts and feature directories.
- `MLA_features_* /`: Extracted feature sets (e.g., `MLA_features_kimianet/`, `MLA_features_resnet50/`).
- `dataset.py`: Data loader logic.
- `create_dataset_csv.py`: Helper to generate the dataset CSV.
- `extract_fv.py`, `uni_extract_fv.py`: Feature extraction scripts.
- `create_patches.py`: Patch extraction script.
- `train_CLAM.py`: Training script for CLAM.
- `simpler_classifiers.py`: Baseline classifiers.
- `avg_std.py`: Computes mean and standard deviation for manual per-cell features to aggregate per patch.
- `conversion_GeoJson.py`: Converts annotation files to GeoJSON for QuPath.
- `heatmap.py`: Evaluates a single WSI and generates a heatmap.
- `merge_features.py`: Concatenates QuPath-extracted features with ResNet50 features.

2. Pipeline Workflow

Step 1: Feature Extraction

Goal: Extract tissue patches from WSI slides (if not already done).

- **Script:** `create_patches.py`
- **Output:** Patch files saved under `patches/`.



Step 2: Feature Extraction

Goal: Convert WSI patches into feature vectors.

- **Script:** `extract_fv.py` or `uni_extract_fv.py`
- **Output:** Feature files in your `MLA_features_*` directory.

Step 3: Manual Feature Extraction

Goal: Extract handcrafted per-cell features from QuPath outputs.

- Using QuPath and the patches calculated previously use QuPath cell detection and then calculate features.

Step 4: Aggregate Manual Features

Goal: Aggregate per-cell features to per-patch statistics.

- **Script:** `avg_std.py`
- **Output:** Per-patch mean and standard deviation features, since the features are calculated using QuPath are for each cell inside the patch it is essential to aggregate them.

Step 5: Merge Features

Goal: Concatenate QuPath manual features with ResNet50 features.

- **Script:** `merge_features.py`

Step 6: Dataset Preparation

Goal: Map slides to labels.

- **Script:** `create_dataset_csv.py`
 - **Input:** Diagnosis file or existing metadata used by the script.
 - **Output:** `dataset.csv` (contains `slide_id` and `label`) and `label_mapping.txt`.
 - **Usage:**
-

```
python create_dataset_csv.py
```

- *Status: Already completed. **dataset.csv** is ready.*

Step 7: Training

Goal: Train the CLAM model.

- **Script:** `train_CLAM.py`
- **Usage:**

```
python train_CLAM.py --csv-path dataset.csv --features-dir  
MLA_features_kimianet/ --model-type clam_sb --n-classes 3 --epochs 20 --  
bag-weight 0.7 --opt adam --cv kfold --n-splits 4
```

- **Key Arguments:**
 - `--csv_path`: Path to `dataset.csv`.
 - `--data_root_dir`: Path to the folder containing feature files.
 - `--model_type`: `clam_sb` (Small) or `clam_mb` (Multi-branch).

Step 8: Evaluation / Heatmap

Goal: Evaluate a single WSI and generate a heatmap.

- **Script:** `heatmap.py`

Attention Heatmap (Pred: 0)

